

Programação Competitiva

Semana 4: Estratégias de resolução de problemas

Alberto Tavares Duarte Neto

10 de Setembro, 2025

Universidade de Brasília

Problemas de otimização

Definição

Um problema de otimização é um problema no qual devem ser feitas escolhas para maximizar (ou minimizar) uma função.

Exemplo. Uma quantidade n de filmes será exibida no cinema hoje. Cada filme tem início s_i e fim f_i . Qual é a maior quantidade de filmes que uma pessoa pode assistir, considerando que não consegue assistir dois filmes simultaneamente?

No exemplo, as escolhas são quais filmes assistir, e a função a ser maximizada é a quantidade de filmes assistidos.

Existem várias técnicas para resolver problemas de otimização.

- Algoritmos gulosos,
- Busca completa (e programação dinâmica),
- Busca binária, no caso em que a função é monotônica,
- Outras técnicas, que não serão discutidas aqui.

Vamos comentar três exemplos de busca completa:

- Gerar todos os subconjuntos de um conjunto,
- Gerar todas as permutações de n elementos,
- Backtracking.

Problema

Dado um conjunto de $n \leq 20$ inteiros, calcule a maior soma de um subconjunto cuja soma seja $\leq X$ para um inteiro X dado.

Exemplo. O conjunto $\{5, 7, 11\}$ tem $2^3 = 8$ subconjuntos:

- $\{\}$,
- $\{5\}$, $\{7\}$, $\{11\}$,
- $\{5, 7\}$, $\{5, 11\}$, $\{7, 11\}$,
- $5, 7, 11$.

Se $X = 13$, a resposta é 12.

Solução recursiva

```
int solve(int i, int soma) {  
    if(i >= n) {  
        if(soma <= X) return soma;  
        return 0;  
    }  
    return max(solve(i+1, soma), solve(i+1, soma+v[i]));  
}
```

Subconjuntos

Seja b um número inteiro e consideremos sua representação binária. Este número representará um subconjunto S da seguinte forma: o i -ésimo elemento do conjunto original estará no subconjunto escolhido se, e somente se, o i -ésimo bit de b está ligado.

Exemplo. O número 110_2 representa o subconjunto $\{7, 11\}$ de $\{5, 7, 11\}$.

Portanto, iterando por todos os números de 0 até $2^n - 1$ e considerando os subconjuntos associados, estaremos iterando por todos os subconjuntos de um conjunto de n elementos.

Solução iterativa

```
for(int b = 0; b < (1 << n); b++) { // (1 << n) = 2^n
    int soma = 0;
    for(int i = 0; i < n; i++) {
        if( (1 << i) & b > 0 ) { // verificando se o i-esimo bit de
            soma += v[i];         // b está ligado
        }
    }
    if(soma <= x) resposta = max(resposta, soma);
}
```

Permutações

Problema

Dado um baralho com cartas numeradas de 1 até $n \leq 10$, encontre todas as ordens possíveis para o baralho.

É possível fazer uma solução recursiva. Apresentamos aqui uma solução utilizando a função **next permutation** do c++.

Solução iterativa

```
vector<int> v(n);  
for(int i = 0; i < n; i++) v[i] = i+1;  
do {  
    for(int i = 0; i < n; i++) {  
        cout << v[i] << " ";  
    }  
    cout << endl;  
} while(next_permutation(v.begin(), v.end()));
```

Definição

Cormen define um **algoritmo guloso** como um algoritmo que toma decisões ótimas localmente, com o intuito de obter uma solução global ótima.

Dificuldades:

- É necessário entender bem as propriedades do problema.
- Pode-se perder muito tempo em uma solução gulosa incorreta.
- Em muitos casos, não existe solução gulosa conhecida para o problema.

Vamos aprender a argumentar a corretude de um algoritmo guloso.